WebGazer Features:

Eye Tracking:

WebGazer uses the already available webcam on laptops, and front camera on mobiles to track the eye movement of the user by calculating the movement of the eye pixels at regular intervals.

It includes a number of computer vision algorithms known as tracker modules. These are used to determine how the information of eye movement should be calculated. The three standard tracker modules that ship with WebGazer.js are .clmtrackr, js_objectdetect, tracking.js. However, other algorithms can also be configured allowing for higher degrees of customization.

Machine Learning:

The library continuously tracks user clicks and mouse movements and then uses the information to self-calibrate itself, improving the accuracy of prediction of eye movements.

It uses a regression model to process the input.

Just like tracker modules, a number of regression modules are available for developers to choose from, like:

- ridge
- weightedRidge
- threadedRidge
- linear

Browser Support

WebGazer uses the Stream API that is supported in most major browsers. These include

- Chrome 47 and above.
- FireFox 44 and above
- Opera 36 and above
- Microsoft Edge 13 and above

Applications:

An efficient eye tracking tools has many applications and it is only limited by the capabilities of the hardware. Some of the significant ones are:

- **e-Commerce -** Online shopping information is one of the goldmines that almost every e-commerce sites track these days using crawlers and other means. But that highly relies on you actually interacting with the website, which might not always be the gaze. With this library, such ecommerce websites will now be able to track where you are looking, which product holds your gaze longer, what are the things you notice first on a product page and what gets ignored.
- **Advertising -** The principles of e-commerce also apply to online advertising in terms of tracking. An advertiser would definitely want to know if people are even looking at his ads, whether they are actually ignoring it or is it catching their eyes. This also helps website owners to place ads better so that they receive more attention. With possibilities of dynamic generation, analysing a gaze pattern over time might even lead to targetted advertising, with a website loading differently for users with different gaze patterns.
- **Gaming -** The ability to track the gaze opens multiple new doors for gaming. A player can analyze their gaze pattern over multiple matches to know where they are going wrong or figure out weak spots. It would be handy if you can figure out your blind spots in Dust II, wouldn't it? Although there are other gaze tracking technologies, the lightness of this makes it ideal for gaming, where performance needs to be optimum. Beyond self analysis and improvement, gaze tracking in gaming can be a completely new method of interaction. Consider this, you are playing a highly detailed single player game and an object catches your attention. Gazing at it ingame brings up detailed information and a zoomed in view of the object. These are only a couple of possibilties and there are endless new ways to game with this.
- **Virtual Reality -** If there is one area that can benefit endlessly from gaze tracking it is Virtual Reality. Even though there aren't really any mainstream websites that are completely VR enabled , mobile VR already tracks the zone you are facing long enough to detect that as a click. With advanced gaze tracking, it is highly likely that simple interactions like scrolling, zooming in, minimising and maximising can be done via gaze changes. Also, since rendering VR can be quite tasking for most processors and GPU's, especially when it is within a browser, it might make sense to restrict high quality renders to the area where the user is looking.

Disadvantages:

Since this tool essentially democratizes tracking, there are several possible concerns:

- **Privacy** - We do not need to tell you explicitly that giving access to your webcam might open doors to hackers, letting them see you and your surroundings, but we still will. Unless you are completely sure of the websites credentials, or a completely new and reliable certificate based on this is released, it would probably be wise to grant access quite rarely.
- **Performance -** Any device, irrespective of how light and resource friendly it is, does have a lifetime. WebGazer essentially takes control of your webcam continuously, constantly recording the frames and analysing them to get a better estimate of your gaze. Unless you have a really high quality webcam, this might lead to an early demise of that device.
- **Accuracy -** Technically, we do look at what we are interacting with online, but it is not always completely precise. WebGazer can at the most get an estimate of the general are we are looking at for quite a while as of now. Even if it could precisely pinpoint the location of the user's gaze, it might not necessarily be the exact thing we are interacting with and that parameter would vary from website to website.

Creating The WebGazer.

Here we create a simple program that uses WebGazer to generate a heatmap of the user over a period of one day

STEP 1. Create a simple HTML Page with a button in it which onclick will call a js function named "stopTracking".

```
<button  onclick="stopTracking()" > Stop Tracking </button>
```

STEP 2. Include the WebGazer.js (https://WebGazer.cs.brown.edu) and simpleheat.js (https://github.com/mourner/simpleheat) in the html file.

```
<script src="WebGazer.js"></script>
<script src="simpleheat.js"></script>
```